

Q1.

A parent process creates three child processes using `fork()`. One child finishes immediately, while the parent continues executing for another 20 seconds without calling `wait()`. What happens to the child that has already finished? Explain your answer.

Q2.

A system administrator observes that a long-running process has been waiting for a considerable amount of time because shorter processes are continuously being selected by the SJF scheduler. Evaluate the fairness of SJF in this situation and suggest a suitable mechanism to overcome the problem. Justify your answer.

Q3.

Identify which OS service (from: program execution, I/O operations, file-system manipulation, communication, error detection, resource allocation, accounting, protection & security) is invoked in each case:

- i. A user transfers a file from a pen drive to the computer.
- ii. Two processes exchange data using pipes.
- iii. The OS records CPU usage for each user.
- iv. A process tries to access memory belonging to another process and is terminated.

Q4.

A programmer writes a program that creates a child process using `fork()`. The parent exits immediately without calling `wait()`, while the child continues running. Will the child stop executing? If not, explain what happens to it.

Q1. Independent Worker (Orphan Process)

A **manager process** creates a **worker process**.

Write a C program that:

- Creates one child process using `fork()`.
- The parent process exits immediately without calling `wait()`.
- The child process:
 - Prints its PID.
 - Sleeps for 5 seconds.
 - Prints its PPID.
 - Exits.

Q2. Delayed Worker (Zombie Process)

Write a C program that:

- Creates one child process using `fork()`.
- The child process prints its PID and exits immediately.
- The parent process waits for 10 seconds before calling `wait()` (or `waitpid()`).
- The parent process then exits.

Q3. Manager and Worker Process

Write a C program that:

- Creates one child process using `fork()`.
- The parent process prints:

Manager is managing the task.

- The child process prints:

Worker is ready to execute.

- Both the parent and child processes print their PID before exiting.